

Islamabad Metro Bus AI System — Backend API

Base URL (dev): `http://localhost:3000`

All request/response bodies are JSON. All timestamps are ISO 8601 strings.

Authentication

Two token types, both JWTs issued via the login endpoints, sent as:

```
Authorization: Bearer <token>
```

Role	Obtained from	Used for
admin	POST <code>/api/login</code>	all admin CRUD + management endpoints
driver	POST <code>/api/driver/login</code>	POST <code>/api/gps/update</code> , POST <code>/api/crowd/update</code> (only for buses assigned to that driver)

The AI module authenticates separately via an API key header instead of a JWT:

```
x-api-key: <AI_API_KEY>
```

Tokens expire after 7 days. Unauthorized requests return `401`.

Auth

POST `/api/login`

Admin login.

Body: { "email": string, "password": string } **200:** { "token": string, "admin": { "id", "name", "email" } } **401:** invalid credentials

POST `/api/driver/login`

Driver login.

Body: { "cnic": string, "password": string } **200:** { "token": string, "driver": { "id", "name", "cnic" } } **401:** invalid credentials or inactive driver

Stations

GET `/api/stations`

Public. List all stations. **200:** { "stations": Station[] }

POST `/api/stations` admin

Body: { "name": string, "latitude": number, "longitude": number } **201:** { "station": Station }

GET `/api/stations/[id]`

Public. **200:** { "station": Station } · **404** if not found

PUT /api/stations/[id] 🔒 **admin**

Body: any subset of { name, latitude, longitude } **200:** { "station": Station }

DELETE /api/stations/[id] 🔒 **admin**

200: { "success": true }

Routes

A `Route` has an ordered list of stations (`busRoutes` , sorted by `sequence`).

GET /api/routes

Public. Includes each route's ordered stations. **200:** { "routes": Route[] }

POST /api/routes 🔒 **admin**

Body: { "name": string, "description"?: string, "stationIds"?: string[] } (`stationIds` order defines the route sequence) **201:** { "route": Route }

GET /api/routes/[id]

Public. **200:** { "route": Route } · **404**

PUT /api/routes/[id] 🔒 **admin**

Body: any subset of { name, description, stationIds } — passing `stationIds` replaces the full station sequence. **200:** { "route": Route }

DELETE /api/routes/[id] 🔒 **admin**

200: { "success": true }

Drivers

All driver endpoints are admin-only (drivers don't manage their own records). Responses never include `passwordHash` .

GET /api/drivers 🔒 **admin**

200: { "drivers": Driver[] }

POST /api/drivers 🔒 **admin**

Body: { "name", "cnic", "phone", "email"?, "password" (min 6 chars) } **201:** { "driver": Driver }

GET /api/drivers/[id] 🔒 **admin**

200: { "driver": Driver } · **404**

PUT /api/drivers/[id] 🔒 **admin**

Body: any subset of { name, phone, email, password, isActive } **200:** { "driver": Driver }

DELETE /api/drivers/[id] 🔒 **admin**

200: { "success": true }

Buses

GET /api/buses

Public. Includes driver (public fields only) and route . **200:** { "buses": Bus[] }

POST /api/buses 🔒 **admin**

Body: { "busNumber": string, "capacity"? : number, "driverId"? : string, "routeId"? : string } **201:** { "bus": Bus }

GET /api/buses/[id]

Public. **200:** { "bus": Bus } · **404**

PUT /api/buses/[id] 🔒 **admin**

Body: any subset of { busNumber, capacity, isActive } **200:** { "bus": Bus }

DELETE /api/buses/[id] 🔒 **admin**

200: { "success": true }

PATCH /api/buses/[id]/assign 🔒 **admin**

Assign/unassign a driver and/or route to a bus. **Body:** { "driverId"? : string | null, "routeId"? : string | null } **200:** { "bus": Bus } (with driver/route included)

Live Data (polling)

The system uses simple REST polling — no WebSockets. Clients (portals, Flutter app) should poll these on an interval (e.g. every 5–10s).

POST /api/gps/update 🔒 **driver (must own the bus)**

Logs a GPS point and triggers rule-based alert evaluation (route deviation + stall/delay detection — see Alerts below). **Body:** { "busId": string, "latitude": number, "longitude": number, "speed"? : number } **201:** { "gpsLog": GPSLog } · **403** if bus isn't assigned to this driver

POST /api/crowd/update 🔒 **driver (must own the bus)**

Body: { "busId": string, "level": "LOW" | "MEDIUM" | "HIGH" } **201:** { "crowdStatus": CrowdStatus } · **403** if bus isn't assigned to this driver

GET /api/live-buses

Public. All active buses with their latest GPS position and crowd level — this is what the passenger portal and admin live map poll. **200:**

```
{
  "buses": [
    {
      "id": "...", "busNumber": "...", "driver": {...}, "route": {...},
      "location": { "latitude": 0, "longitude": 0, "speed": 0, "recordedAt": "..." } | null,
      "crowdLevel": "LOW" | "MEDIUM" | "HIGH" | null
    }
  ]
}
```

GET /api/eta?busId=...&stationId=...

Public. Returns an ETA for a bus arriving at a station.

Resolution order:

1. If an AI prediction exists for this bus+station pair and is **< 2 minutes old**, use it (`source: "ai"`).
2. Otherwise fall back to a rule-based estimate: haversine distance from the bus's last known GPS point to the station, divided by the bus's last reported speed (or a 25km/h default) (`source: "rule-based"`).

200 (ai source): { "busId", "stationId", "etaMinutes", "delayMinutes", "source": "ai", "predictedAt" } **200 (rule-based source):** { "busId", "stationId", "distanceKm", "etaMinutes", "source": "rule-based", "basedOn": {...} } **404:** station not found, or no GPS data yet for the bus

Alerts

Alerts are created automatically by rule-based detection on every `gps/update` call:

- **ROUTE_DEVIATION:** bus is >0.8km from its assigned route's path. Auto-resolves once back within range.
- **DELAY:** bus's last 3 GPS points all show speed <2km/h spanning ≥5 minutes. Auto-resolves once moving again.

GET /api/alerts?status=OPEN  **admin**

`status` query param is optional (`OPEN` | `ACKNOWLEDGED` | `RESOLVED`). **200:** { "alerts": Alert[] }
(includes `bus`)

PATCH /api/alerts/[id]  **admin**

Manually acknowledge/resolve an alert. **Body:** { "status": "OPEN" | "ACKNOWLEDGED" | "RESOLVED" } **200:** { "alert": Alert }

AI Predictions

GET /api/predictions?busId=...&stationId=...

Public. Both query params optional (filter). Returns most recent 50, newest first. **200:** { "predictions": AIPrediction[] }

POST /api/predictions  **x-api-key header (AI service only)**

Called by the Python AI module to push a new prediction. **Body:** { "busId": string, "stationId"?: string, "predictedEtaMinutes"?: number, "predictedDelayMinutes"?: number } **201:** { "prediction": AIPrediction }

Activity Logs

GET /api/logs  **admin**

Most recent 100 admin actions (create/update/delete/assign/alert changes), newest first. Includes the acting admin's public info. **200:** { "logs": ActivityLog[] }

Error shape

All error responses: { "error": string } with an appropriate HTTP status (400 validation, 401 unauthorized, 403 forbidden, 404 not found).

Environment variables (.env , see .env.example)

Var	Purpose
DATABASE_URL	Postgres connection string
JWT_SECRET	signs/verifies admin & driver JWTs
AI_API_KEY	shared secret for the AI module's POST /api/predictions